

第五章: 回溯算法

Design and Analysis of Computer Algorithms

Xiu-Jun GONG

College of Intelligence and Computing, Tianjin University

Email: gongxj@tju.edu.cn

April, 2025

1 定义、表示与方法

- 解空间
- 表示方法
- 状态空间
- 状态空间搜索

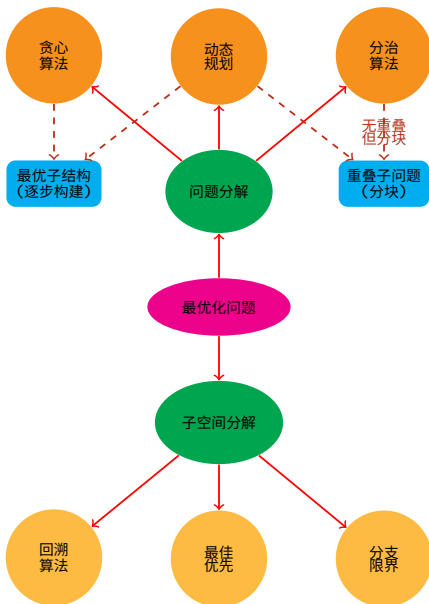
2 两船装箱问题

3 0/1 背包问题

4 最大团问题

5 旅行商问题

动机



形式化定义

- ① 计算问题定义为关系 $R \subseteq I \times S$, 其中:
 - I 是输入实例集合
 - S 是候选解集合
 - 实例 x 的解是满足 $(x, s) \in R$ 的任意元素 $s \in S(x)$
- ② 对于输入实例 $x \in I$, 解空间 $S(x)$ 是满足约束的所有有效解集合:

$$S(x) = \{s \in S \mid (x, s) \in R\}$$

- ③ 问题 P 的全局解空间定义为:

$$SP = \{s \in S(x) \mid x \in I \text{ 且 } (x, s) \in R\}$$

关键特征:

- **判定问题**: 解空间为二元集合 (如 {是, 否})
- **搜索问题**: 解空间包含所有有效输出 (如数的质因数集合)
- **最优化问题**: 解空间包含所有可行解, 最优解满足:

$$f(s^*) = \min_{s \in S(x)} f(s) \quad \text{或} \quad f(s^*) = \max_{s \in S(x)} f(s)$$

示例:

- **旅行商问题 (TSP)**:
 - **输入**: 带权图 G
 - **解空间**: G 中所有哈密顿回路
 - **最优解**: 总权重最小的回路

元组表示法

- 解可表示为 n 维元组 $(x_1, x_2, \dots, x_n)$
 - x_i 取自有限集合 S_i
 - 根据维度是否固定可分为：
 - 定长元组 (FLT): 所有解元组维度相同
 - 变长元组 (VLT): 解元组维度可变
- 解空间的形式化表示：
 - FLT: $X = \{(x_1, x_2, \dots, x_n) | x_i \in S_i\}$
 - VLT: $X = \{X^1, X^2, \dots, X^k\}$, 其中 $X^j = \{(x_{j1}, x_{j2}, \dots, x_{jk})\}$

八皇后问题

在标准国际象棋棋盘上放置八个皇后，使得任意两个皇后不能互相攻击

	1	2	3	4	5	6	7	8
1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			

Figure 1: 8 皇后问题

- 解可表示为 8 维元组 $(x_1, \dots, x_8)$ ，其中 x_i 表示第 i 行皇后的列号
- 解空间大小: 8^8
- 约束条件: $\forall i, j, x_i \neq x_j$ 且 $|x_i - x_j| \neq |j - i|$

子集和问题

问题描述： 在正整数集合 $S = \{w_1, w_2, \dots, w_n\}$ 中寻找和为 M 的子集

示例: $M = 31$, $n = 4$, $W = (11, 13, 24, 7)$, 存在解 $11+13+7=31$
和 $24+7=31$

- 解的两种表示方法：

FLT 表 示	$(x_1, x_2, \dots, x_n)$, 其中 $x_i = 1$ 表示 选中元素 w_i	$(1, 1, 0, 1)$ $(0, 0, 1, 1)$	$O(2^n)$
------------	--	----------------------------------	----------

VLT 表 示	$(j_1, j_2, \dots, j_k)$, 其中 j_i 为选中元素 下标	$(1, 2, 4)$ $(3, 4)$	$O(2^n)$
------------	---	-------------------------	----------

定义

问题状态 解空间中的一个点，可验证是否满足目标条件

初始状态（根结点） 开始搜索的起始状态

解状态 从当前状态到根结点的路径构成有效解

目标状态 解空间中满足最终目标的状态

问题的**状态空间**可定义为五元组 $(V, E, v_0, \mathcal{S}, \mathcal{A})$:

- V : **问题状态集合**
- E : **状态间转移关系集合**
- $v_0 \in V$: 表示初始状态的根结点
- $\mathcal{S} \subseteq V$: 目标状态集合
- $\mathcal{A}$: **状态转移函数**，生成有效子结点

每个状态 $v \in V$ 可表示：

- **部分解**（如组合问题中的元素子集）
- **完整解**（如排序问题中的有效排列）
- **决策路径**（如回溯算法中的选择序列）

状态空间的数学结构形式：

- 树结构：状态空间树
- 图结构
- 向量空间
- 集合论表示

四皇后问题状态空间树

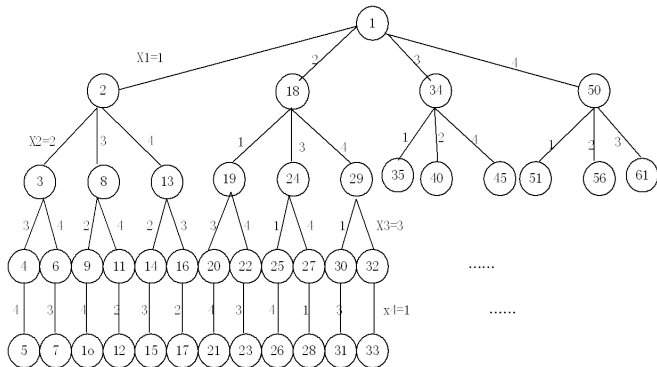


Figure 2: 四皇后问题状态空间树

- 结点编号表示 DFS 遍历顺序
- x_i 表示第 i 行皇后列号, i 对应树的深度
- 此结构称为排列树

子集和问题的 FLT 表示

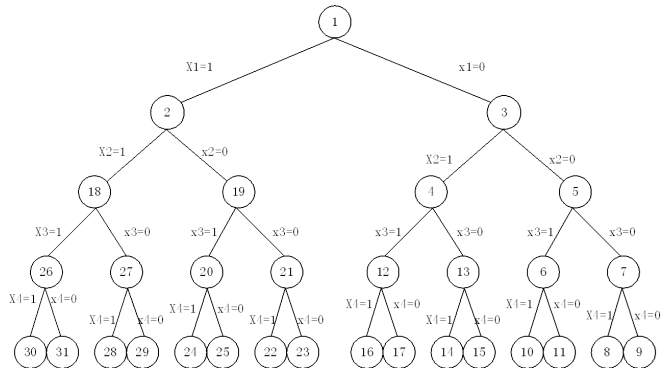


Figure 3: FLT 表示的状态空间树：M=31, n=4, W=(11,13,24,7)

- 结点编号表示 DFS 遍历顺序
- x_i 表示元素选择标志，i 对应树的深度
- 此结构称为**幂集树**

子集和问题的 VLT 表示

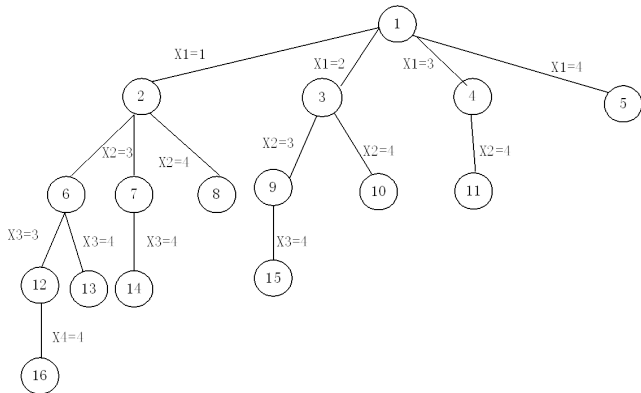


Figure 4: VLT 表示的状态空间树: $M=31$, $n=4$, $W=(11,13,24,7)$

- 结点编号表示 BFS 遍历顺序
- x_i 表示元素在集合中的序号

搜索策略

- 问题求解需要遍历状态空间树
- 结点在遍历中有三种状态：
 - 活动结点 (live node): 已到达但未完全展开
 - 死亡结点 (died node): 所有子结点已展开
 - 扩展结点 (E-Node): 当前正在展开子结点的活动结点
- 常用搜索策略：
 - BFS: 广度优先搜索
 - DFS: 深度优先搜索
 - **回溯算法**: DFS + 剪枝函数
 - 分支限界: 树搜索 + 限界 + 结点选择策略
 - 最佳优先搜索: 优先扩展最有潜力的结点

回溯算法原理

回溯算法是一种通过**深度优先搜索**结合**限界函数**来探索解空间的算法策略。

① 状态空间树表示

回溯算法将问题抽象为一棵**隐式树形结构**，其中树的深度对应决策步骤，宽度对应每一步的可能选择。

② 递归试探与回溯

通过递归逐步构建解，并评估其成为最优解的可能性。若当前选择导致**无效路径**（如违反约束条件），或**无法产生更优解**时则回溯到上一步。

③ 约束与限界函数

- **约束函数**：在搜索过程中实时检查当前路径是否违反问题约束（如重复访问城市），若违反则剪枝。
- **限界函数**：针对优化问题（如最短路径），若当前路径的代价已劣于已知最优解，则提前终止该路径的探索。

回溯算法通用框架

Algorithm 1: 回溯算法框架

```
1 函数 回溯算法 (int n)
2    k=1;
3    while k > 0 do
4        forall  $x[k] \in$  可行值集合 do
5            /* 基于已选值  $x[1] \dots x[k-1]$  生成候选值 */
6            if 约束条件  $B(x[1], \dots, x[k])$  不成立 then
7                if 当前路径构成有效解 then
8                    输出解  $(x[1], \dots, x[k])$ ;
9                    k=k+1 /* 继续扩展下一层 */
10                else
11                    k = k-1 /* 回溯到上一层 */
12
13 /* 解存储在  $X(1:n)$  中 */
```

子集和问题的限界函数

- 基础限界条件：当部分解的总和加上剩余元素总和仍小于目标值时剪枝

$$\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) < M \quad (1)$$

- 增强限界条件（元素非递减排序）：

$$\sum_{i=1}^k W(i)X(i) + W(k+1) > M \quad (2)$$

带限界的子集和算法

Algorithm 2: 子集和问题限界算法

- 1 定义当前部分和 $s = \sum_{i=1}^{k-1} W(i)X(i)$;
- 2 定义剩余元素和 $r = \sum_{i=k}^n W(i)$ (假设 $s + r \geq M$) ;

/* 扩展左子结点

*/

- 3 **if** $s + W(k) + W(k+1) > M$ **then**

- 4 停止扩展左子树 ;

- 5 $r = r - W(k)$;

- 6 转向右子树扩展 ;

- 7 **else**

- 8 置 $X(k)=1$;

- 9 更新 $s = s + W(k)$;

- 10 更新 $r = r - W(k)$;

- 11 将当前结点设为扩展结点 ;

/* 扩展右子结点

*/

- 12 **if** $s + r - W(k) < M$ 或 $s + W(k+1) > M$ **then**

- 13 停止扩展

- 14 **else**

- 15 置 $X(k)=0$

示例: $M=30$, $n=6$, $W=(5,10,12,13,15,18)$

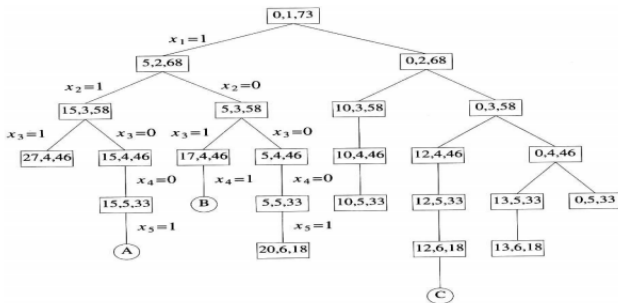


Figure 5: 带限界的状态空间树

- 结点标注格式: (当前和 s , 决策层 k , 剩余和 r)
- 圆形结点表示有效解
- 剪枝后总结点数降为 23 个

回溯算法总结

① 暴力枚举与剪枝结合

- 回溯本质是**系统性枚举**，通过递归遍历所有可能解（状态空间树），通过**剪枝**（约束函数和限界函数）动态排除无效路径，避免穷举。

② 状态空间树的隐式表示

- 问题解空间被抽象为一棵**隐式树形结构**，树的深度对应决策步骤，宽度对应每一步的候选选择。

③ 约束与限界函数的双重剪枝

- 约束函数**：实时检查路径合法性（如是否重复访问城市），若违反则剪枝。
- 限界函数**：针对优化问题（如最短路径），若当前路径代价劣于已知最优解，则剪枝。

④ 典型应用场景

- 组合问题：生成所有子集、排列或组合（如子集和问题）。
- 约束满足：八皇后、图着色等需满足特定条件的解。
- 优化问题：0/1 背包、旅行商问题（TSP）等需在可行解中寻找最优解

⑤ 效率与局限性

- 优点**：通过剪枝显著减少无效计算，相比纯枚举更高效。
- 缺点**：最坏时间复杂度仍为指数级。

1 定义、表示与方法

2 两船装箱问题

- 问题定义
- 解决方案
- 回溯解法

3 0/1 背包问题

4 最大团问题

5 旅行商问题

问题描述

- 给定两艘容量分别为 c_1 和 c_2 的货船，以及 n 个集装箱，其重量为 $(w_1, \dots, w_n)$ ，满足

$$\sum_{i=1}^n w_i \leq c_1 + c_2$$

- 需判定是否存在一种装载方案，使得所有集装箱均可装入两艘货船且不超过载。

注意：

- 当 $\sum_{i=1}^n w_i \geq c_1 + c_2$ 时，该问题等价于子集和问题
- 当 $c_1 = c_2$ 时，该问题等价于划分问题

基本原理解析

- 将第一艘货船尽可能接近其容量装载
- 剩余集装箱装入第二艘货船

问题转化为选择集装箱子集，使其总重量尽可能接近 c_1 的最优化问题：

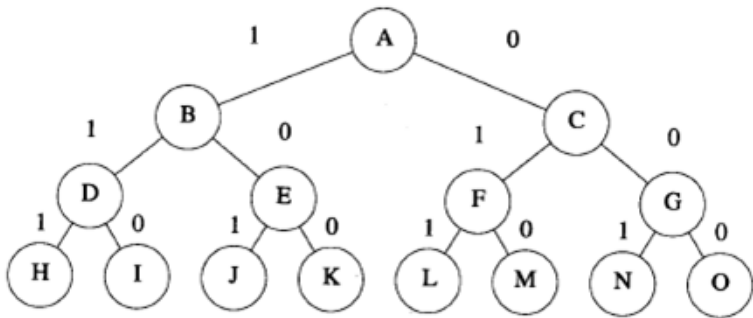
$$\max \sum_{i=1}^n w_i x_i$$

约束条件：

$$\sum_{i=1}^n w_i x_i \leq c_1$$

其中 $x = (x_1, \dots, x_n)$ ($x_i = 1$ 表示装载第 i 个集装箱，否则为 0)

状态空间树示例



- 参数: $n = 4$, $c_1 = 12$, $w = [8, 6, 2, 3]$
- 叶结点未显示

限界函数

将集装箱按重量降序排列，假设当前扩展结点为第 $i - 1$ 层，定义：

$$cw = \sum_{j=1}^{i-1} x_j w_j \quad \text{已装载集装箱总重量}$$

$$r = \sum_{j=i}^n w_j \quad \text{未装载集装箱总重量}$$

$$bestw = \quad \text{当前最优总重量}$$

- **限界条件 1**：若 $cw + w_i > c_1$ ，则剪枝（无法装载第 i 个集装箱）
- **限界条件 2**：若 $cw + r \leq bestw$ ，则剪枝（剩余集装箱无法改善当前最优解）
- 可同时应用两种限界条件

代码实现

```
1 template<class T>
2 T MaxLoading(T w[], T c, int n, int bestx
   [])
3 { // Return best loading and its value.
4   Loading<T> X;
5   // initialize X
6   X.x = new int [n+1];
7   X.w = w;
8   X.c = c;
9   X.n = n;
10  X.bestx = bestx;
11  X.bestw = 0;
12  X.cw = 0;
13  // initial r is sum of all weights
14  X.r = 0;
15  for (int i = 1; i <= n; i++)
16    X.r += w[i];
17  X.maxLoading(1);
18  delete [] X.x;
19  return X.bestw;
20 }
```

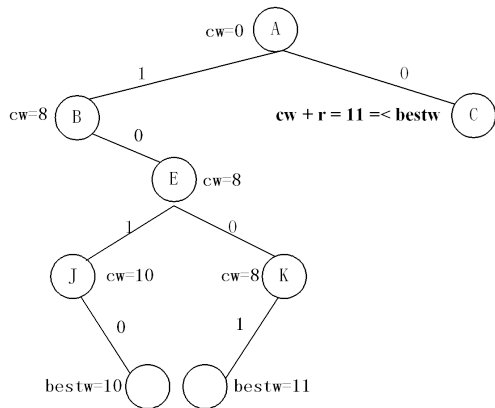
```
1 template<class T>
2 void Loading<T>::maxLoading(int i)
3 { // Search from level i node.
4   if (i > n) { // at a leaf
5     for (int j = 1; j <= n; j++)
6       bestx[j] = x[j];
7     bestw = cw; return; }
8   // check subtrees
9   r -= w[i];
10  if (cw + w[i] <= c) { // try x[i] = 1
11    x[i] = 1;
12    cw += w[i];
13    maxLoading(i+1);
14    cw -= w[i]; }
15  if (cw + r > bestw) { // try x[i] = 0
16    x[i] = 0;
17    maxLoading(i+1); }
18  r += w[i];
19 }
```

思考：

- 展开左子树时，为何没有检查限界条件 2？
- 展开右子树时，为何没有检查限界条件 1？

带限界的状态空间树

实例: $n = 4$, $c_1 = 12$, $w = [8, 6, 2, 3]$



最优解: $x = [1, 0, 0, 1]$

Outline

- 1 定义、表示与方法
- 2 两船装箱问题
- 3 0/1 背包问题
 - 回溯法
- 4 最大团问题
- 5 旅行商问题

限界

假设物品按 p/w 降序排列，结点 $k - 1$ 为 E-结点，定义：

$$cp = \sum_{j=1}^{k-1} x_j p_j$$

当前装包利润

$$rp = \sum_{j=k}^n p_j$$

剩余物品总利润（松限界）

$$rp = \sum_{j=k}^{m-1} p_j + \frac{cleft}{w[m]} * p[m]$$

部分松弛后的总利润（紧限界）

$$bestp =$$

当前最大利润

限界条件：若 $cp + rp \leq bestp$ 则限界

代码

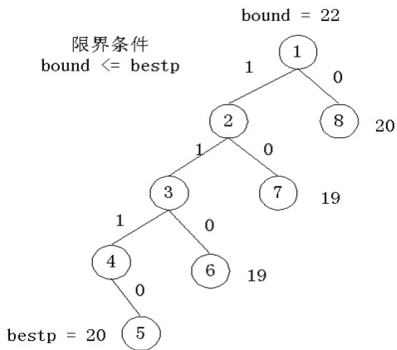
```
1 template<class Tw, class Tp>
2 void Knap<Tw, Tp>::Knapsack(int i)
3 { // Search from level i node.
4     if (i > n) { // at a leaf
5         bestp = cp;
6         return; }
7     // check subtrees
8     if (cw + w[i] <= c) { // try x[i] = 1
9         cw += w[i];
10        cp += p[i];
11        Knapsack(i+1);
12        cw -= w[i];
13        cp -= p[i]; }
14    if (Bound(i+1) > bestp) // try x[i] =
15        Knapsack(i+1);
16 }
```

```
1 template<class Tw, class Tp>
2 Tp Knap<Tw, Tp>::Bound(int i)
3 { // Return upper bound on value of
4   // best leaf in subtree.
5   Tw cleft = c - cw; // remaining
6                       // capacity
7   Tp b = cp; // profit bound
8   // fill remaining capacity
9   // in order of profit density
10  while (i <= n && w[i] <= cleft) {
11      cleft -= w[i];
12      b += p[i];
13      i++;
14  }
15  // take fraction of next object
16  if (i <= n) b += p[i]/w[i] * cleft;
17  return b;
18 }
```

假设物品已按效益密度 $\left(\frac{p[i]}{w[i]}\right)$ 排序

带限界的状态空间树

实例: $n = 4$, $c = 7$, $p = [9, 10, 7, 4]$, $w = [3, 5, 2, 1]$



解: $x = [1, 1, 1, 0]$

Outline

- 1 定义、表示与方法
- 2 两船装箱问题
- 3 0/1 背包问题
- 4 最大团问题**
 - 问题定义
 - 回溯法求解
- 5 旅行商问题

问题陈述

子图 $G' = \langle V', E' \rangle$ 是无向图 $G = \langle V, E \rangle$ 的**完全子图**，当且仅当 $V' \subset V$ 且对 $\forall u \in V', \forall v \in V', (u, v) \in E' \subset E$

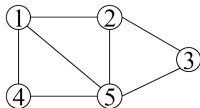
团是 G 的完全子图且不被其他完全子图包含

独立顶点集是 G 的无边子图且不被其他独立集包含

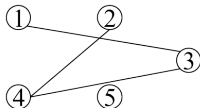
最大团是图中最大尺寸的团

最大独立顶点集是图中最大尺寸的独立集

$\{1, 2\}$ 是完全子图但不是团
 $\{1, 2, 5\}, \{1, 4, 5\}, \{2, 3, 5\}$ 是最大团
 $\{2, 4\}$ 是最大独立顶点集



$\{1, 2\}$ 是空子图但不是独立集
 $\{1, 2, 5\}$ 是独立顶点集
 $\{1, 2, 5\}$ 也是最大独立顶点集



限界

- 目标：寻找图 G 的最大团
- 使用固定长度元组 $X = x[1..n]$ ($x[i] = 1$ 表示包含顶点 i) 表示解空间
- 状态空间树为子集树
- 限界条件：
 - ① 从根到结点 i 的顶点不能构成完全子图
 - ② 根到结点 i 的顶点数加上剩余顶点数不超过当前最优解 $bestn$

代码

```
1 int AdjacencyGraph::MaxClique(int v[])
2 { // Return size of largest clique.
3   // Return clique vertices in v[1:n].
4   // initialize for maxClique
5   x = new int [n+1];
6   cn = 0;
7   bestn = 0;
8   bestx = v;
9
10  // find max clique
11  maxClique(1);
12
13  delete [] x;
14  return bestn;
15 }
```

```
1 void AdjacencyGraph::maxClique(int i)
2 { // Backtracking code to compute largest
   clique.
3   if (i > n) { // at leaf
4     // found a larger clique, update
5     for (int j = 1; j <= n; j++)
6       bestx[j] = x[j];
7     bestn = cn;
8     return; }
9   // see if vertex i connected to others
10  // in current clique
11  int OK = 1;
12  for (int j = 1; j < i; j++)
13    if (x[j] && a[i][j] == NoEdge) {
14      // i not connected to j
15      OK = 0;
16      break; }
17
18  if (OK) { // try x[i] = 1
19    x[i] = 1; // add i to clique
20    cn++;
21    maxClique(i+1);
22    x[i] = 0;
23    cn--; }
24
25  if (cn + n - i > bestn) { // try x[i] =
    0
26    x[i] = 0;
27    maxClique(i+1); }
28 }
```

Outline

- 1 定义、表示与方法
- 2 两船装箱问题
- 3 0/1 背包问题
- 4 最大团问题
- 5 旅行商问题**
 - 问题定义
 - 回溯法

问题陈述

- **旅行商问题** (Traveling Salesman Problem, TSP) 是组合优化领域里经典的 NP 完全问题。其目标是给定城市列表及每对城市间的距离，寻找**最短可能路线**，该路线需**精确访问每个城市一次**并返回**起始城市**。
- TSP 的输入可以表示为一个图结构 $G = (V, E)$ ，其中：
 - $V = \{v_1, v_2, \dots, v_n\}$ 为顶点集合 (表示城市)
 - E 为边集合 (表示城市间的路径)
 - 每条边 $e = (v_i, v_j)$ 关联一个非负权重 $a_{ij} \in \mathbb{R}^+$
- TSP 的目标是寻找一个**哈密尔顿回路** (Hamiltonian cycle) C ，满足：
 - C 访问每个顶点 **恰好一次**
 - C 是闭合回路 (起点 = 终点)
 - 回路的总权重 (总距离) **最小**
- **关键特性**
 - **对称性**: 若 $a_{ij} = a_{ji}$ 对所有 i, j 成立, 称为对称 TSP (Symmetric TSP); 否则为非对称 TSP (Asymmetric TSP)。
 - **起点无关性**: 在对称 TSP 问题中, 最优回路的总长度与路径的起点选择无关。
 - **度量性**: 若距离满足三角不等式 ($d_{ik} \leq a_{ij} + a_{jk}$), 称为度量 TSP, 允许近似算法。

限界

- 定义 $x = x[1..n]$ (x_i 为路径中第 i 个结点的序号) 表示解
- 目标函数为:

$$\sum_{i=1}^n a_{x_i, x_{i+1}} \text{ where } (x_{n+1} = x_1)$$

- 状态空间树为排列树
- 限界条件:
 - ① 若不存在连接 $x[i]$ 与 $x[i-1]$ 的边, 则限界
 - ② 若从根到 $x[i]$ 的路径距离超过当前最优解 $bestc$ (已知最短路径), 则限界

代码

```

1 void AdjacencyWDigraph<T>::tSP(int i)
2 { // Backtracking code for traveling salesperson.
3   if (i == n) { // at parent of a leaf
4     // complete tour by adding last two edges
5     if (a[x[n-1]][x[n]] != NoEdge &&
6         a[x[n]][1] != NoEdge &&
7         (cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc ||
8          bestc == NoEdge)) { // better tour found
9       for (int j = 1; j <= n; j++)
10         bestx[j] = x[j];
11       bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1];
12     }
13   else { // try out subtrees
14     for (int j = i; j <= n; j++)
15       // is move to subtree labeled x[j] possible?
16       if (a[x[i-1]][x[j]] != NoEdge &&
17           (cc + a[x[i-1]][x[j]] < bestc ||
18            bestc == NoEdge)) { // yes
19         // search this subtree
20         Swap(x[i], x[j]);
21         cc += a[x[i-1]][x[i]];
22         tSP(i+1);
23         cc -= a[x[i-1]][x[i]];
24         Swap(x[i], x[j]);
25       }
26 }

```

```

1 template<class T>
2 T AdjacencyWDigraph<T>::TSP(int v
   [])
3 { // Traveling salesperson by
   backtracking.
4   // Return cost of best tour,
   return tour in v[1:n].
5   // initialize for tSP
6   x = new int [n+1];
7   // x is identity permutation
8   for (int i = 1; i <= n; i++)
9     x[i] = i;
10  bestc = NoEdge;
11  bestx = v; // use array v to
   store best tour
12  cc = 0;
13
14  // search permutations of x[2:
   n]
15  tSP(2);
16
17  delete [] x;
18  return bestc;
19 }

```



```

1 // 全局变量
2 int n; // 城市数量
3 int[][] dist; // 距离矩阵 (dist[i][j]表示城市i
    到j的距离)
4 boolean[] visited; // 访问标记数组
5 int bestc = INF; // 当前最优解 (初始化为无穷大)
6 int[] bestpath; // 最优路径
7 int[] current_path; // 当前路径
8 // 主函数
9 void TSP_Backtrack() {
10     visited[0] = true; // 固定起点为城市0 (假设
        城市编号0~n-1)
11     current_path.add(0); // 初始化当前路径
12     backtrack(0, 1, 0); // 初始调用: 当前城市0
        , 已访问1个城市, 路径长度0
13 }
14
15 //递归回溯函数
16 void backtrack(int current_city, int count
    , int current_length) {
17     if (count == n) { // 所有城市已访问, 需返回
        起点
18         int total_length = current_length +
            dist[current_city][0];
19         if (total_length < bestc) {
20             bestc = total_length;
21             bestpath = copy(current_path);
                // 保存当前最优路径
22         }
23         return;
24     }
25
26     // 遍历未访问城市
27     for (int i = 0; i < n; i++) {
28         if (!visited[i]) {
29             // 剪枝: 若当前路径+到i的距离已超过最优
                解, 则跳过
30             if (current_length + dist[
                current_city][i] >= bestc
                ) {
31                 continue; // [[9]]
32             }
33             // 选择城市i
34             visited[i] = true;
35             current_path.add(i);
36             // 递归扩展
37             backtrack(i, count + 1,
                current_length + dist[
                current_city][i]);
38             // 回溯
39             visited[i] = false;
40             current_path.remove_last(); //
                恢复路径
41         }
42     }
43 }

```

Coming up: Branch & Bound
Algorithm